PlussMa Distribution Documentation

PlussMa Contributors

Mar 01, 2026

CONTENTS

1 Quickstart **3**
- 1.1 Include Path . . . 3
- 1.2 Sample In One Line . . . 3
- 1.3 Keep Shape And Dtype . . . 3
- 1.4 Evaluate Densities And CDFs . . . 3
- 1.5 Gaussian Mixture . . . 3
- 1.6 Intended Style . . . 4

2 API Reference **5**
- 2.1 Entry Point . . . 5
- 2.2 Core Functions . . . 5
- 2.3 Notes On Dtypes . . . 6

3 Distribution Catalog **7**
- 3.1 Core Continuous Families . . . 7
- 3.2 Shape / Tail Families . . . 7
- 3.3 Symmetric / Bounded Families . . . 8
- 3.4 Count / Discrete Families . . . 8
- 3.5 Derived / Physics / Special Cases . . . 8
- 3.6 Parameter Conventions . . . 9

4 Build Outputs **11**
- 4.1 Install Doc Dependencies . . . 11
- 4.2 Build HTML . . . 11
- 4.3 Build PDF . . . 11
- 4.4 Relevant Code Artifacts . . . 11
- 4.5 Validation Path . . . 11

The legacy Matlab `randraw.m` surface has been retired. Probability distributions now live in the C++ API as `pluss::distr`, available directly from:

```
#include <pluss/pluss.h>
```

This manual is organized around how users actually consume the layer:

- start with `quickstart` for the short-path workflow
- use `api_reference` when you need the function surface
- use `distribution_catalog` when you need names, aliases, and parameter conventions
- use `build_outputs` when you want to build the docs or the example binary locally

CHAPTER

ONE

QUICKSTART

1.1 Include Path

```
#include <pluss/pluss.h>
```

The distribution layer is exposed as `pluss::distr`. No extra include is needed when PlussMa is part of the build.

1.2 Sample In One Line

```
auto x = pluss::distr::sample("normal", {0.0, 1.0}, {1024});
auto y = pluss::distr::sample("arcsin", {1024});
auto z = pluss::distr::sample("vonmises", {0.0, 3.0}, {1024});
```

1.3 Keep Shape And Dtype

```
auto like = pluss::zeros({2, 4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32));

auto gamma_like = pluss::distr::sample_like("gamma", {2.0}, like);
auto signs_like = pluss::distr::sample_like("rademacher", like);
```

1.4 Evaluate Densities And CDFs

```
auto grid = pluss::make_series(std::vector<double>{-2.0, -1.0, 0.0, 1.0, 2.0}, {5});

auto normal_pdf = pluss::distr::pdf("normal", grid, {0.0, 1.0});
auto t_cdf = pluss::distr::cdf("t", grid, {5.0});
auto nig_logpdf = pluss::distr::logpdf("nig", grid, {2.0, 0.5, 0.0, 1.0});
```

1.5 Gaussian Mixture

```
auto weights = pluss::make_series(std::vector<double>{0.65, 0.35}, {2});
auto means = pluss::make_series(std::vector<double>{-1.0, 2.0}, {2});
auto stddevs = pluss::make_series(std::vector<double>{0.5, 1.25}, {2});

auto mix = pluss::distr::mixture_normal(
```

(continues on next page)

(continued from previous page)

```
    weights,
    means,
    stddevs,
    {4096},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float64),
    23ULL,
    true);
```

The last argument switches the base normal generator from pseudo-random draws to Sobol-driven Gaussian draws.

1.6 Intended Style

The user-facing model is:

- keep everything under `pluss::distr`
- pass the distribution name plus a short parameter list
- let `sample_like` carry shape and dtype when you already have a target tensor
- stay on `Series` throughout instead of bouncing between helper wrappers

CHAPTER

TWO

API REFERENCE

2.1 Entry Point

All distribution functionality is available from:

```
pluss::distr
```

That alias resolves to `pluss::subsp::stat::distr`, but user code should generally prefer the shorter public path.

2.2 Core Functions

2.2.1 supports

```
bool pluss::distr::supports(std::string_view name) noexcept;
```

Returns whether a distribution name or alias is recognized.

2.2.2 sample

```
pluss::Series pluss::distr::sample(
    std::string_view name,
    std::span<const double> params,
    std::span<const std::int64_t> sizes,
    pluss::Config cfg = {},
    std::uint64_t seed = 0);
```

Convenience overloads also exist for:

- `initializer_list<double>` parameters
- `initializer_list<std::int64_t>` sizes
- zero-parameter distributions via `sample(name, sizes, cfg, seed)`

2.2.3 sample_like

```
pluss::Series pluss::distr::sample_like(
    std::string_view name,
    std::span<const double> params,
    const pluss::Series& like,
    std::uint64_t seed = 0);
```

Convenience overloads also exist for:

- `initializer_list<double>` parameters
- zero-parameter distributions via `sample_like(name, like, seed)`

`sample_like` preserves the shape, platform, and dtype of `like`.

2.2.4 pdf

```
pluss::Series pluss::distr::pdf(
    std::string_view name,
    const pluss::Series& x,
    std::span<const double> params = {});
```

2.2.5 logpdf

```
pluss::Series pluss::distr::logpdf(
    std::string_view name,
    const pluss::Series& x,
    std::span<const double> params = {});
```

2.2.6 cdf

```
pluss::Series pluss::distr::cdf(
    std::string_view name,
    const pluss::Series& x,
    std::span<const double> params = {});
```

2.2.7 mixture_normal

```
pluss::Series pluss::distr::mixture_normal(
    const pluss::Series& weights,
    const pluss::Series& means,
    const pluss::Series& stddevs,
    std::span<const std::int64_t> sizes,
    pluss::Config cfg = {},
    std::uint64_t seed = 0,
    bool use_sobol = false);
```

This samples from an `n`-component Gaussian mixture.

- `weights`, `means`, and `stddevs` must have the same length
- `weights` are normalized internally
- `use_sobol=true` uses Sobol-driven Gaussian draws

2.3 Notes On Dtypes

- density-style functions return floating-point `Series`
- discrete samplers use natural integral output where appropriate
- `sample_like` is the shortest path when you want the result to match an existing tensor contract

CHAPTER

THREE

DISTRIBUTION CATALOG

The catalog below follows the retired `randraw.m` naming scheme so older usage strings still map cleanly into the C++ layer.

3.1 Core Continuous Families

- `normal`, `gaussian`, `gauss`, `norm`
- `lognorm`, `lognormal`, `cobbdouglas`, `antilognormal`
- `uniform`, `unif`
- `exp`, `exponential`
- `cauchy`, `lorentz`, `caushy`
- `laplace`, `doubleexponential`, `doubleexp`, `bilateralexponential`, `bilateralexp`
- `logistic`
- `studentst`, `t`
- `normaltrunc`, `normaltruncated`, `gausstrunc`
- `vonmises`

3.2 Shape / Tail Families

- `beta`, `powerfunction`, `powerfunc`
- `gamma`
- `weibull`, `wbl`, `frechet`
- `pareto`
- `pareto2`, `lomax`
- `burr`, `fisk`
- `gompertz`
- `planck`
- `gig`
- `gh`
- `nig`

3.3 Symmetric / Bounded Families

- alpha
- anglit
- arcsin
- cosine
- halfcosine, hcosine, hcos
- hyperbolicsecant, hsecant, hsec
- quadratic, quad, quadr
- tri, triangular
- u, ushape
- semicirc, semicircle, wigner

3.4 Count / Discrete Families

- bernoulli, bern
- binom, binomial
- geom, geometric, furry
- poisson, po
- negativebinomial, negbinomial, negbinom, pascal
- hypergeom, hypergeometric
- bessel
- chisquare, chisq, chi2
- chisqnc, chisqnoncentral, chisqnoncentr, chi2noncentral
- skellam
- rademacher
- yule, yulesimon
- zeta, zipf

3.5 Derived / Physics / Special Cases

- chi
- erlang
- f, fdistribution, fdistrib, fdistr, fdist, fdis
- fnc, fnoncentral, fnoncentr
- ig, inversegauss, invgauss
- wald
- rayl, rayleigh

- `maxwell`
- `nakagamim`, `nakagami-m`, `nakagami`
- `rice`, `rician`

3.6 Parameter Conventions

Selected conventions carried over from the legacy surface:

- `beta`: `[a, b]` or `[m, n, a, b]`
- `gamma`: `[a]` or `[m, b, a]`
- `t`: `[nu]` or `[nu, chi, eta]`
- `chi`: `[nu]` or `[a, b, nu]`
- `weibull`: `[shape, scale]` or legacy `[theta, alpha, beta]`
- `normaltrunc`: `[a, b, mu, sigma]`
- `hypergeom`: `[N, M, n]`
- `nig`: `[alpha, beta, mu, delta]`
- `gh`: `[lambda, alpha, beta, mu, delta]`
- `gig`: `[lambda, chi, psi]`

When in doubt, use `pluss::distr::supports(name)` first and then keep the parameter list minimal.

CHAPTER

FOUR

BUILD OUTPUTS

4.1 Install Doc Dependencies

```
python3 -m pip install -r requirements.txt
```

4.2 Build HTML

```
make html
```

The HTML output lands in:

```
_build/html
```

4.3 Build PDF

```
make pdf
```

This expects a working LaTeX toolchain with `latexmk`.

4.4 Relevant Code Artifacts

The docs describe the production surface implemented in:

- `PlussMa/Stats/distributions/distr.cpp`
- `PlussMa/include/pluss/subsp/stat/distr_fwd.h`
- `PlussMa/include/pluss/ma/stats.h`
- `PlussMa/examples/Numerical/distributions_demo.cpp`

4.5 Validation Path

The runtime coverage for the documented surface is in:

```
PlussMa/Stats/distributions/test/distr_test.cpp
```